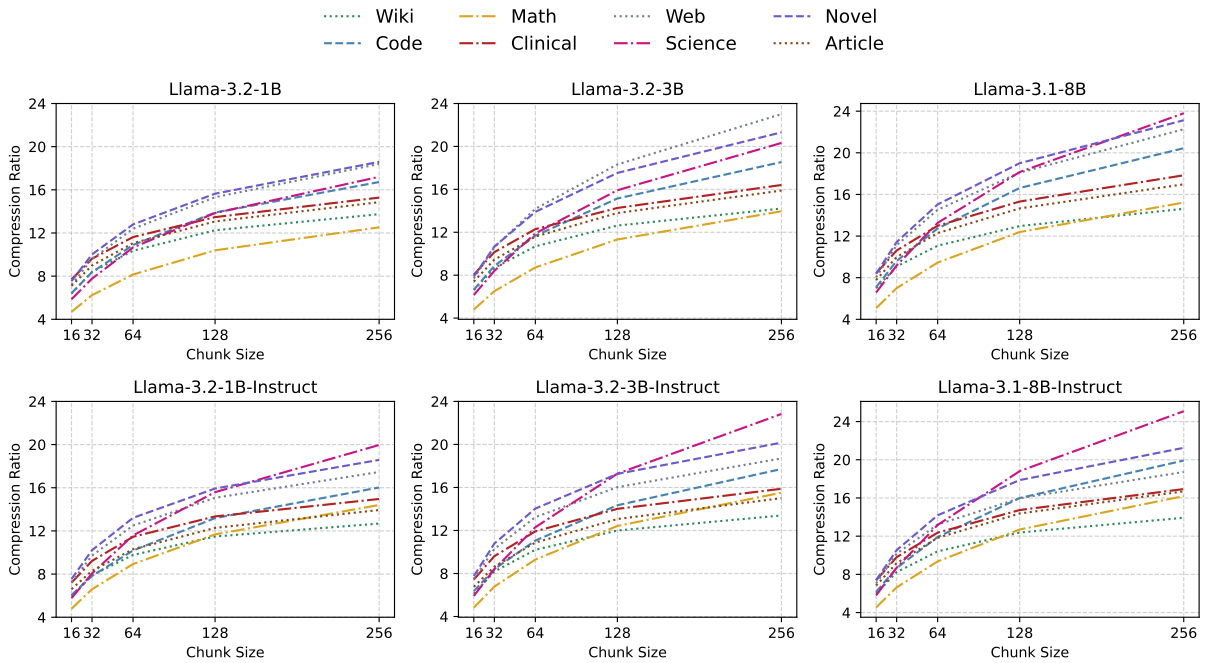

Figure 5: Compression ratio comparison across different Llama models. Larger models achieve higher compression ratios, demonstrating the advantage of increased model capacity in leveraging contextual information. Instruction-tuned variants generally perform slightly worse than their base counterparts on most datasets but achieve better compression on question-answering datasets.

threshold, the additional context provided by larger chunk sizes yields diminishing returns in terms of compression performance.

5.5 Impact of Model Size on Compression Peformance

One of the most straightforward yet effective ways to improve compression efficiency is to increase the model size. In this subsection, we analyze the impact of model size on compression rate by examining Fig. 5, which illustrates that larger models consistently achieve higher compression rates across both base and instruction-tuned variants. To provide a more detailed comparison, we further include Fig. 6, which expands the analysis to six models: Apple1B, AMD1B, Llama1B, Llama3B, Llama8B, and Qwen14B.

The experimental results reveal a clear upward trend in compression ratio with increasing model size. Across all conditions, the smallest models exhibit the lowest compression rates, while the largest models, such as Llama8B and Qwen14B, demonstrate significantly higher compression performance. This pattern holds across different datasets and configurations, reinforcing the notion that larger models benefit from enhanced predictive capabilities, which allow them to reduce redundancy more effectively. Due to hardware constraints, our experiments are limited to models up to 14B parameters. We believe that 14B is sufficient to cover most LLM compression applications, as it already represents a large-scale model. However, we acknowledge that exploring even larger models remains an interesting direction for future research.

5.6 Impact of Dataset Scale on Compression Performance

Dataset scale is another crucial factor influencing compression efficiency. In this subsection, we investigate whether increasing the size of LLM-generated synthetic data affects compression performance. To this end, we evaluate both traditional and LLM-based compressors under different scales of the Wiki dataset.

The results are given in Fig. 7. Traditional methods such as Huffman coding, Arithmetic coding, FSE coding, and Gzip exhibit no significant change in compression rate, indicating that their underlying encoding mechanisms do not benefit from increased data availability. More advanced methods like LZMA and Zstd-22 show a slight improvement in compression rate with larger datasets, but the growth remains modest. In contrast, NN-based compressors such as TRACE and PAC demonstrate both superior compression performance and a slightly higher rate of improvement as dataset size increases. However, none of these methods come close to matching the compression efficiency achieved by the proposed approach.

A particular observation is that the proposed method maintains a consistent compression rate regardless of dataset scale. The stability of the proposed method across different dataset scales suggests potential applications in scenarios where data volume fluctuates, such as real-time streaming or incremental data storage.

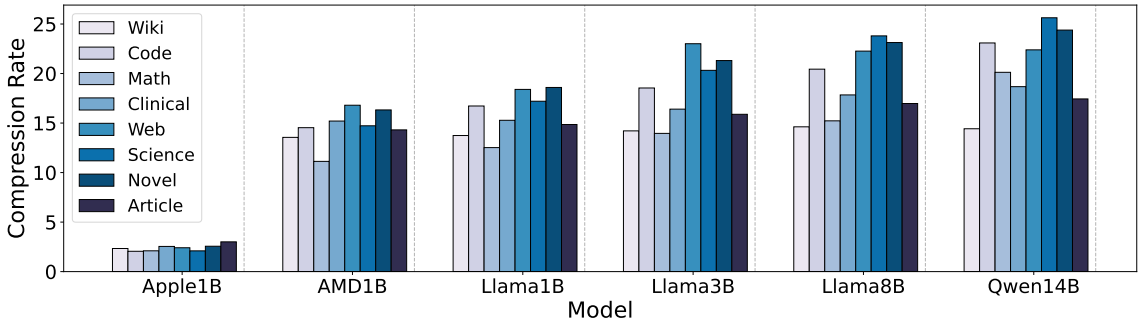

Figure 6: Model scale vs. compression ratio. Larger models consistently demonstrate higher compression ratios, indicating that increasing model size enhances predictive capabilities and further reduces redundancy.

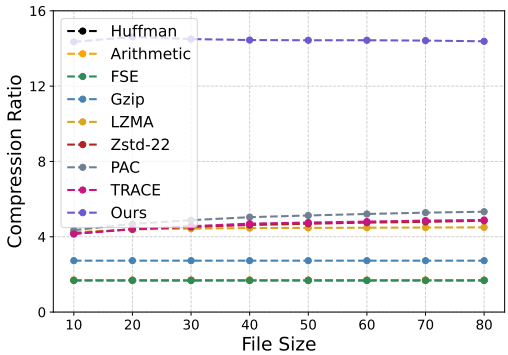

Figure 7: Compression ratio variations with increasing dataset size. Traditional methods remain stable, while neural compressors show slight improvement, but the proposed method consistently outperforms all approaches.

5.7 Comparison between LLMs with and without Fine-tuning

5.7.1 Instruction tuning. Instruction tuning is an indispensable post-training procedure for existing deployed LLMs, which aims to enable LLMs to align with human preference data and not exhibit unsafe behaviors. As instruction tuning also fine-tunes the token distributions of the base LLMs, in this subsection, we conduct an investigation into the influence of instruction tuning on the compression capabilities of LLMs for the text data.

The experimental results reveal that instruction-tuned variants generally exhibit a slightly lower compression ratio compared to their base model across most datasets. This phenomenon can be attributed to the differences in training objectives between pretraining and instruction tuning. During the initial training phase of large language models, the objective function is designed to minimize the cross-entropy loss between the predicted token distribution and the true labels, which aligns naturally with the goal of compression (minimizing entropy). However, instruction tuning introduces a different optimization objective that alters the token distribution, potentially reducing its effectiveness for compression.

Unlike pretraining, instruction tuning focuses on aligning model outputs with human preferences rather than purely maximizing predictive accuracy. This process involves reinforcement learning with human feedback (RLHF) or supervised fine-tuning on instruction-following datasets, where the model is explicitly optimized to generate responses that are coherent, contextually relevant, and stylistically appropriate rather than merely predicting the most probable next token. These factors can lead to a token distribution that is less compressible, as the model is encouraged to generate structured, human-like responses rather than the statistically optimal token sequences for entropy minimization.

One example supporting this hypothesis is the observation that instruction-tuned models achieve a consistent increase in compression efficiency on science and math datasets across different model sizes. Since these datasets primarily consist of structured question-answer pairs, their underlying probability distributions are more aligned with the instruction-tuning objective, which emphasizes coherent and structured responses. This alignment likely results in improved token predictability within these domains, leading to better compression rates.

5.7.2 Specific Domain Fine-tuning. To verify the compressibility of domain-specific models on their respective domains, we evaluate compression ratios using specialized models on math and code datasets. For the math dataset, we select Qwen2.5-Math-1.5B and Rho-Math-1B. For the code dataset, we use Qwen2.5-Coder-1.5B and DeepSeek-Coder-1.3B-Instruct. To ensure fairness, all selected models have sizes ranging from approximately 1B to 1.5B.

Table 8 presents the compression performance of general models and two domain-specific models on math and code tasks. While larger models such as Qwen14B and Llama8B generally achieve higher compression ratios, the domain-specific models exhibit significantly better compression efficiency. On the math dataset, when compared to similarly sized 1B models, Qwen2.5-Math-1.5B and Rho-Math-1B achieve noticeably higher compression ratios. Remarkably, Rho-Math-1B reaches a compression ratio comparable to Qwen14B, despite being 14 times smaller in size. For the code dataset, Qwen2.5-Coder-1.5B attains a compression ratio similar to Llama8B. However, DeepSeek-Coder-1.3B performs comparably to the general-purpose Llama1B model, which we suspect may be due to insufficient training. This suggests that models explicitly

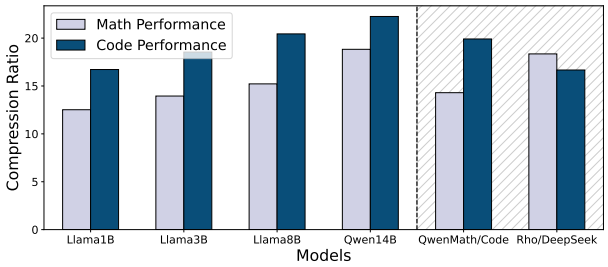

Figure 8: Performances of domain-specific models on Math and Code tasks. Domain-specific models consistently achieve higher compression ratios within their respective domains.

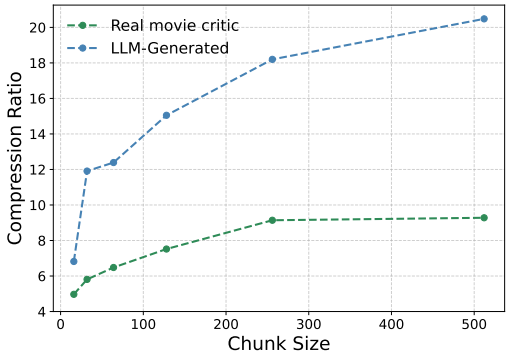

Figure 9: Compression ratio comparison between human-generated and LLM-generated text. LLMs achieve significantly higher compression ratios on LLM-generated data, with the gap widening as chunk size increases.

trained or fine-tuned for specific domains can better capture the structural patterns and redundancies inherent in domain-specific data, leading to improved compression rates.

5.8 Predictability of LLMs on Human vs. LLM-Generated Data

In this subsection, we study the predictability of LLMs for human-generated data and LLM-generated data. We evaluate the compression ratios of LLMs under different chunk sizes for the two categories of text data. Specifically, we run Llama-3.2-1B on real movie critics extracted from imdb and LLM-generated movie critics.

The results are shown in Figure 9. We can find that under different chunk sizes, LLMs demonstrate significantly higher compression ratios on LLM-generated data over human data. More interestingly, along with the increase in the chunk size, the compression ratios increase even further. Therefore, in large-scale data compression, as the ratios of LLM-generated data keep growing, LLM-based compressors will demonstrate increasingly significant compression capabilities compared to traditional compressors.

6 Related Work

6.0.1 Traditional Compression. Traditional lossless compression methods leverage entropy coding and dictionary-based techniques to efficiently reduce data size while preserving exact information. Huffman coding [24] assigns variable-length codes based on symbol frequency, ensuring shorter codes for more common symbols. Arithmetic coding [51] provides better compression by representing entire sequences as fractional numbers, achieving near-optimal entropy efficiency. Dictionary-based methods like Gzip [15] and LZMA [58] utilize Lempel-Ziv compression, replacing repeated substrings with references to previous occurrences. Zstd [10], an advanced variant, improves upon Gzip with faster compression and decompression speeds while maintaining high compression ratios.

6.0.2 Neural Network-based Compressors. The use of neural networks as predictors for lossless compression has been widely explored, particularly in conjunction with arithmetic coding [3, 12, 19]. NNCP [3] employs transformers to estimate probability distributions, while TRACE [34] introduces a lightweight transformer variant to accelerate the compression process. Some works start using foundation models for lossless compression recently [14, 23, 40]. Studies leveraging models such as LLaMA[38] and Chinchilla [14] demonstrate promising results, particularly in compressing previously unseen modalities—for instance, large language models trained on text effectively compress images and audio. However, none of these studies specifically address *the domain of synthetic data*, leaving a gap in exploring how such models perform in artificial data compression scenarios.

7 Conclusion

As LLM-generated data continues to grow exponentially, effective lossless compression of LLM-generated data has become a critical challenge in text data management. In this work, we conducted the first systematic investigation into compressing LLM-generated text. We demonstrated that traditional lossless compressors struggle to effectively compress LLM-generated data due to the irregularity and complexity of LLM-generated data. Unlike previous machine-generated data, LLM-generated data lacks structured redundancy and appears more similar to human-generated data, posing critical challenges to lossless compression. To mitigate the issue, we proposed leveraging the predictive capabilities of LLMs originating from next-token prediction. Our extensive experiments with 14 LLMs and 8 datasets revealed that LLM-based prediction methods achieve over 20× compression, far surpassing Gzip's 3× compression. Furthermore, we show that this approach generalizes across different model scales and dataset categories, underscoring the robustness and versatility of LLM-based compression.

References

[1] Ayad Akhtar. 2024. The Role of AI in Creative Writing: A Case Study on McNeal. *The Atlantic* (2024). https://www.theatlantic.com/magazine/archive/2024/11/mcneal-play-akhtar-downey-artificial-intelligence/679949/.
[2] Yushi Bai, Jiajie Zhang, Xin Lv, Linzhi Zheng, Siqi Zhu, Lei Hou, Yuxiao Dong, Jie Tang, and Juanzi Li. 2024. LongWriter: Unleashing 10,000+ Word Generation from Long Context LLMs. *arXiv preprint arXiv:2408.07055* (2024).
[3] Fabrice Bellard. 2023. NNCP: Neural Network Compression Program. https://bellard.org/nncp/. Accessed: 2024-02-19.
[4] Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel M. Ziegler, Jeffrey Wu, Clemens Winter, Christopher Hesse, Mark Chen, Eric Sigler, Mateusz Litwin, Scott Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya